MATERIAL DIDÁCTICO

UNIDAD DIDÁCTICA 01

Relación 4

Completar código, paso a paso

Dos ejercicios largos de repaso para trabajar sobre programas ya empezados, con dos archivos editables por ejercicio y comprobaciones tras cada bloque.

01 / CSV Y COPIAS

Lectura, validación, altas, modificación, borrado y restauración

02 / BINARIO Y XML

Serialización, cambio en memoria, persistencia y exportación DOM

JAVA 17+ · 2.º DAM · SIN RANDOMACCESSFILE

Cómo trabajar esta relación

Un simulacro no se resuelve de golpe: se completa por etapas pequeñas.

El objetivo es interpretar y continuar código existente

Los programas compilan desde el inicio, aunque los métodos con TODO aún devuelvan valores provisionales o muestren un aviso en el menú. Se va completando una operación cada vez.

1 Reglas de trabajo

A **Abre una carpeta por ejercicio**

Cada ejercicio tiene su propio Main.java. No mezcles sus clases ni archivos.

B **Edita solamente dos archivos**

En el ejercicio 1: Main.java + GestorInscripciones.java. En el ejercicio 2: Main.java + RepositorioEquipos.java. Las clases del modelo ya están resueltas.

C **Termina un TODO y compruébalo**

Respeta el orden del enunciado. Primero lectura, luego búsqueda y después operaciones que modifican datos. Hay casos de prueba concretos en cada ejercicio.

D **Respeta la separación de responsabilidades**

Main solicita datos y muestra mensajes. Los métodos auxiliares no hacen System.out ni leen teclado; devuelven objetos, valores o lanzan excepciones.

2 Puesta en marcha

Desde la terminal situada dentro de la carpeta del ejercicio:

```
javac -encoding UTF-8 *.java
java Main
```

Requisitos: JDK 17 o superior. Sin Maven ni bibliotecas externas. La carpeta datos debe estar junto a los archivos Java y debes ejecutar el programa desde esa carpeta.

Importante: trabaja con copias de los ficheros de prueba. Los datos se modificarán al realizar operaciones. No se permite RandomAccessFile.

Ejercicio 01 · Gestor de inscripciones

Duración orientativa: 75–90 minutos | CSV, CRUD, UTF-8 y seguridad.

Contexto y datos

Un centro guarda inscripciones a talleres en datos/inscripciones.csv con formato id;persona;taller;plazas. Ya hay cuatro registros: 101 Lucía, 102 Mario, 103 Nuria y 104 Andrés.

1 Pasos que debes completar

1. **Cargar el CSV · TODO 1.1**
   Usa Files.readAllLines y UTF-8. Omite la cabecera, separa con split(";", -1), convierte ID y plazas y rechaza filas inválidas o IDs repetidos mediante IOException.
2. **Buscar por ID · TODO 1.2**
   Recorre la lista; devuelve el objeto encontrado o null. Ejecuta ya las opciones 1 y 2 del menú para comprobar que la lectura funciona.
3. **Modificar plazas · TODO 1.3**
   Rechaza plazas negativas o ID inexistente. Si hay cambio, haz copia del CSV y vuelve a guardar todos los registros.
4. **Eliminar · TODO 1.4**
   Construye una lista sin el ID solicitado. Si no aparece, no crees copia ni reescribas el fichero.
5. **Restaurar · TODO 1.5**
   Recupera inscripciones.csv.bak si existe. Devuelve boolean; la confirmación se pedirá desde Main.
6. **Completar menú · TODO 1.6**
   En Main.java, conecta opciones 4, 5 y 6. Pide números y confirmación S/N, y muestra al usuario el resultado.

Ya te damos resuelto: el modelo Inscripcion, el guardado del CSV, el alta (opción 3), la creación de copia, los casos 1 y 2 del menú y la gestión general de excepciones.

Ejercicio 02 · Archivo de equipos

Duración orientativa: 75–90 minutos | Serializable, .dat y DOM XML.

Contexto y datos

El centro conserva cuatro equipos en datos/equipos.dat: micrófono (201), cables & adaptadores (202), cámara (203) y altavoz (204). Es una lista de objetos Equipo serializada, no un fichero de texto.

2 Pasos que debes completar

1. **Cargar lista binaria · TODO 2.1**
   Si el .dat no existe, devuelve lista vacía. Si existe, usa ObjectInputStream, readObject(), valida que sea List<?>, comprueba los tipos de cada elemento y evita IDs duplicados.
2. **Guardar lista binaria · TODO 2.2**
   Crea la carpeta si falta y utiliza ObjectOutputStream para escribir la lista completa. No uses APPEND.
3. **Cambiar unidades · TODO 2.3**
   Busca un equipo de la lista; si existe y las unidades son válidas, cámbialas solo en memoria. No guardes aquí.
4. **Exportar XML · TODO 2.4**
   Construye un documento con raíz <equipos>; cada elemento lleva atributo id e hijos nombre y unidades. Usa createElement, setTextContent y Transformer.
5. **Completar menú · TODO 2.5**
   Conecta la modificación, la recarga desde disco y la exportación XML en Main.java (opciones 3, 5 y 6).

Ya te damos resuelto: Equipo implements Serializable, un fichero .dat de prueba, la búsqueda por ID, la opción de listado y el guardado explícito desde Main.

Comprobaciones finales

No basta con que compile: cada operación debe producir el resultado esperado.

1 Pruebas del gestor CSV

1. **Comprobación 1**
   En la opción 1 aparecen 4 inscripciones; buscar ID 102 devuelve Mario.
2. **Comprobación 2**
   Cambiar ID 101 a 5 plazas modifica solo ese registro.
3. **Comprobación 3**
   Buscar ID 999 o introducir -1 plazas no altera el archivo.
4. **Comprobación 4**
   Eliminar ID 104 conserva los demás; restaurar con S recupera el registro borrado.
5. **Comprobación 5**
   Reiniciar el programa permite volver a leer los cambios guardados.

2 Pruebas del archivo binario y XML

1. **Comprobación 1**
   La lista inicial muestra 4 equipos; buscar ID 202 encuentra «Cables & adaptadores».
2. **Comprobación 2**
   Cambiar ID 201 a 12 unidades y recargar sin guardar devuelve las 5 originales.
3. **Comprobación 3**
   Cambiar a 12, guardar y reiniciar conserva las 12 unidades.
4. **Comprobación 4**
   El XML contiene cuatro elementos equipo y escapa & como & sin perder las tildes.

Alcance: estos dos casos reúnen lectura y escritura, validación, CRUD, copias, serialización y XML. El bloque de JSON/Jackson de la Relación 3 queda fuera del simulacro obligatorio para mantener solo dos proyectos manejables.

Entrega: los dos proyectos con sus archivos Java y los datos generados. No adjuntes archivos .class; conserva los .java, .csv, .dat, .bak y .xml que correspondan.